

Analyse: Zeiterfassung und Projektzeit-Verteilung

Erstellt: 16. Februar 2026
Autor: Automatische Code-Analyse
Status: Aktuelle Ist-Analyse mit identifizierten Schwachstellen und Inkonsistenzen

1. Architekturübersicht

1.1 Datenmodell

Die Zeiterfassung basiert auf folgenden Prisma-Modellen:

Modell	Datei	Funktion
TimeEntry	schema.prisma:524	Kernentität – ein Clock-In/Clock-Out-Paar
ProjectTimeAllocation	schema.prisma:551	Nachträgliche Verteilung eines TimeEntry auf mehrere Projekte
Employee	schema.prisma:2764	Mitarbeiterstammdaten mit Compliance-Feldern
Project	schema.prisma:406	Projekt, dem Zeit zugeordnet werden kann
OvertimeBalance	schema.prisma:602	Jährliche Überstunden-/Überzeit-Saldierung
ComplianceViolation	schema.prisma:622	Verstösse gegen ArG/ArGV 1
CostCenter	Referenziert in TimeEntry	Kostenstelle (optional)
Location	Referenziert in TimeEntry	Arbeitsort (optional)

1.2 API-Endpunkte

Zeiterfassung (/api/time/):

Endpunkt	Methode	Auth	Funktion
/clock-in	POST	User	Stempeln (mit optionalem Projekt, Ort, Beschreibung)
/clock-out	POST	User	Ausstempeln (mit optionaler Pausenzeit)
/start-pause	POST	User	Pause starten
/end-pause	POST	User	Pause beenden
/current	GET	User	Aktiver Eintrag
/my-entries	GET	User	Eigene Einträge (Datumsfilter)
/my-entries/:id	PUT	User	Eigenen Eintrag bearbeiten
/my-entries/:id	DELETE	User	Eigenen Eintrag löschen
/manual-entry	POST	Admin	Manuellen Eintrag erstellen
/:id	PUT	Admin	Beliebigen Eintrag ändern

Endpunkt	Methode	Auth	Funktion
<code>/:id</code>	DELETE	Admin	Beliebigen Eintrag löschen
<code>/user/:userId</code>	GET	Admin	Einträge eines Users
<code>/logged-in-users</code>	GET	User	Aktuell eingestempelte Mitarbeiter

Projektzeit-Verteilung (`/api/project-time-allocations/`):

Endpunkt	Methode	Auth	Funktion
<code>/time-entry/:timeEntryId</code>	GET	User	Allokationen eines Eintrags
<code>/time-entry/:timeEntryId</code>	POST	User	Allokationen setzen/ersetzen
<code>/:allocationId</code>	DELETE	User	Einzelne Allokation löschen
<code>/stats</code>	GET	User	Statistik nach Projekt

Reports (`/api/reports/`):

Endpunkt	Methode	Auth	Funktion
<code>/my-summary</code>	GET	User	Eigene Zusammenfassung
<code>/user-summary/:userId</code>	GET	Admin	User-Zusammenfassung
<code>/all-users-summary</code>	GET	Admin	Alle User
<code>/project-summary/:projectId</code>	GET	Admin	Projekt-Zusammenfassung
<code>/project-time-by-user</code>	GET	Admin	Projektzeit pro User
<code>/time-bookings</code>	GET	Admin	Detaillierte Zeitbuchungen
<code>/user-time-bookings/:userId</code>	GET	Admin	User-Zeitbuchungen
<code>/overtime-report</code>	GET	Admin	Überstunden-Report

1.3 Ablauf: Clock-In → Clock-Out → Allokation

- Mitarbeiter stempelt ein (clock-in)
 - TimeEntry mit status=CLOCKED_IN, optionalem projectId
 - Compliance-Check: Ruhezeit (11h Minimum)
- Optionale Pausen (start-pause / end-pause)
 - Status wechselt zwischen CLOCKED_IN ↔ ON_PAUSE
 - pauseMinutes wird kumuliert
- Mitarbeiter stempelt aus (clock-out)
 - status=CLOCKED_OUT
 - Compliance-Checks: Tagesarbeitszeit, Pausen, Wochenstunden
 - OvertimeBalance wird aktualisiert
 - Budget-Update (falls Projekt mit Budget)
- Nachträgliche Projektzeit-Verteilung (optional)
 - ProjectTimeAllocation-Einträge erstellen
 - Stunden müssen den Netto-Arbeitsstunden entsprechen ($\pm 0.1h$ Toleranz)

2. Identifizierte Schwachstellen

2.1 KRITISCH: Inkonsistente Verwendung von `userId` vs. `employeeId`

Das gravierendste Problem im gesamten Zeiterfassungssystem.

Das `TimeEntry`-Modell hat zwei Felder:

- `userId` (als DEPRECATED markiert)
- `employeeId` (primäre Relation)

Der **Time Controller** (`time.controller.ts`) arbeitet korrekt mit `employeeId` — er holt über `getEmployeeId(userId)` die Employee-ID.

Aber der **Report Controller** (`report.controller.ts`) filtert an vielen Stellen direkt nach `userId`:

Report-Funktion	Zeile	Filter	Status
<code>getMySummary</code>	L18	<code>{ userId }</code>	FALSCH – <code>userId</code> ist DEPRECATED in <code>TimeEntry</code>
<code>getUserSummary</code>	L98	<code>{ userId }</code>	FALSCH
<code>getAllUsersSummary</code>	L176	<code>{ userId: user.id }</code>	FALSCH
<code>getOvertimeReport</code>	L405	<code>{ userId: user.id }</code>	FALSCH
<code>getProjectTimeByUser</code>	L475	<code>{ userId: user.id, projectId }</code>	FALSCH
<code>getUserTimeBookingsReport</code>	L764	<code>{ userId }</code>	FALSCH
<code>getDetailedTimeBookings</code>	L601	<code>where.userId = userId</code>	FALSCH

Konsequenz: Wenn `userId` in der `TimeEntry`-Tabelle NULL ist (was bei neueren Einträgen der Fall sein kann, da nur `employeeId` befüllt wird), liefern diese Reports **keine Ergebnisse** zurück. Die Berichte sind unvollständig oder leer.

2.2 KRITISCH: Pausenzeit wird in Reports inkonsistent berücksichtigt

Die Helper-Funktion `calculateWorkHours` in `report.controller.ts` (Zeile 8):

```
const calculateWorkHours = (clockIn: Date, clockOut: Date | null): number => {
  if (!clockOut) return 0;
  return (clockOut.getTime() - clockIn.getTime()) / (1000 * 60 * 60);
};
```

Diese Funktion berechnet die **Brutto-Arbeitszeit** (ohne Pausenabzug). Sie wird in folgenden Reports verwendet:

Report	Pausenabzug?
<code>getMySummary</code>	NEIN – Bruttostunden
<code>getUserSummary</code>	NEIN – Bruttostunden

Report	Pausenabzug?
<code>getAllUsersSummary</code>	NEIN – Bruttostunden
<code>getProjectSummary</code>	NEIN – Bruttostunden
<code>getOvertimeReport</code>	NEIN – Überstunden basieren auf Bruttostunden!
<code>getProjectTimeByUser</code>	NEIN – Bruttostunden
<code>getAttendanceByMonth</code>	NEIN – Bruttostunden
<code>getDetailedTimeBookings</code>	JA – <code>netHours</code> korrekt berechnet
<code>getUserTimeBookingsReport</code>	JA – Pausenabzug korrekt

Konsequenz: Die meisten Summary- und Overtime-Reports zeigen zu hohe Stundenzahlen an. Der Überstunden-Report (`getOvertimeReport`) berechnet Überstunden auf Basis der Bruttostunden, was zu falschen Überstunden-Berechnungen führt.

2.3 KRITISCH: Überstunden-Berechnung (OvertimeBalance) berücksichtigt Pausen nicht

In `compliance.service.ts` (Zeile 310ff.):

```
let totalHours = 0;
entries.forEach((entry: any) => {
  if (entry.clockOut) {
    const duration = (entry.clockOut.getTime() - entry.clockIn.getTime()) / (1000
* 60 * 60);
    totalHours += duration; // KEINE Berücksichtigung von pauseMinutes!
  }
});
```

Die `updateOvertimeBalance`-Funktion berechnet Überstunden anhand der Bruttostunden. Da Pausen nicht abgezogen werden, werden **systematisch zu viele Überstunden** in die `OvertimeBalance`-Tabelle geschrieben.

Gleiches Problem in `checkWeeklyHoursViolation` (Zeile 100ff.) – die wöchentlichen Gesamtstunden enthalten Pausen, was zu **fälschlichen Compliance-Verstößen** führen kann.

2.4 HOCH: Duale Projekt-Zuordnung – Überschneidung `TimeEntry.projectId` und `ProjectTimeAllocation`

Das System hat **zwei Mechanismen** für die Projekt-Zuordnung:

1. **`TimeEntry.projectId`** – ein einzelnes Projekt, das beim Clock-In gewählt wird
2. **`ProjectTimeAllocation`** – nachträgliche Verteilung auf mehrere Projekte

Probleme:

- Es gibt **keine Validierung**, ob `ProjectTimeAllocation`-Einträge mit dem `projectId` des `TimeEntry` übereinstimmen
- Ein `TimeEntry` kann `projectId = "Projekt A"` haben, aber die Allokationen können zu "Projekt B" und "Projekt C" gehen
- Reports verwenden mal `TimeEntry.projectId` (z.B. `getProjectSummary`, `getMySummary`), mal `projectTimeAllocations` (z.B. `getDetailedTimeBookings`)
- **Budget-Update** (`budgetUpdate.service.ts`) verwendet nur `TimeEntry.projectId` und ignoriert `ProjectTimeAllocation` komplett!

Konsequenz:

- Wenn ein Mitarbeiter beim Clock-In Projekt A wählt und anschliessend die Zeit auf Projekte B und C aufteilt, wird das Budget nur für Projekt A aktualisiert (falsch)
- Reports zeigen je nach Endpunkt unterschiedliche Projekt-Zuordnungen

2.5 HOCH: Keine Validierung auf überlappende Zeiteinträge

Die `clockIn`-Funktion prüft nur, ob der Mitarbeiter aktuell eingestempelt ist (`status: 'CLOCKED_IN'`). Es gibt **keine Prüfung** auf:

- Überlappende manuelle Einträge (Admin kann beliebige Zeiten eintragen)
- Einträge, die in der Zukunft liegen
- Einträge mit `clockOut` vor `clockIn` (nur bei `createTimeEntry` geprüft, nicht bei `updateTimeEntry`)

2.6 HOCH: Admin-Löschung ohne Audit-Trail

Die Admin-Löschfunktion `deleteTimeEntry` (Zeile 476) löscht Einträge unwiderruflich ohne:

- Protokollierung (kein Action-Trigger)
- Soft-Delete-Pattern (obwohl andere Module es verwenden)
- Rückrechnung des Budgets
- Rückrechnung der OvertimeBalance
- Rückrechnung der Compliance-Violations

Konsequenz: Gelöschte Einträge hinterlassen inkonsistente Budget- und Überstunden-Daten.

2.7 MITTEL: Wöchentliche Höchstarbeitszeit – Race Condition im Compliance-Check

`checkWeeklyHoursViolation` prüft auf eine bestehende Violation für die Woche:

```
const existingViolation = await prisma.complianceViolation.findFirst({
  where: { employeeId, type: 'MAX_WEEKLY_HOURS', date: { gte: weekStart, lte: weekEnd } }
});
```

Wenn in derselben Woche ein weiterer Clock-Out erfolgt, wird die bestehende Violation **nicht aktualisiert** – die `actualValue` bleibt beim alten Wert. Bei konkurrenten Clock-Outs (extrem unwahrscheinlich, aber möglich) könnte auch eine doppelte Violation entstehen.

2.8 MITTEL: Tägliche Höchstarbeitszeit ignoriert Pausen

`checkDailyHoursViolation` berechnet:

```
const duration = (clockOut.getTime() - clockIn.getTime()) / (1000 * 60 * 60);
if (duration > 12.5) { ... }
```

Die 12.5-Stunden-Grenze wird auf Basis der Bruttodauer berechnet. Eine 13-stündige Anwesenheit mit 2 Stunden Pause (= 11 Stunden Nettoarbeit) würde fälschlicherweise als Verstoss gewertet.

2.9 MITTEL: CostCenter-Feld wird nicht genutzt

`TimeEntry` hat ein `costCenterId`-Feld, aber:

- Die `clockIn`-Funktion akzeptiert es nicht im `req.body`
- Es wird in keinem Report berücksichtigt
- Es gibt keine UI-Komponente zur Auswahl
- Es wird in der `createTimeEntry`-Admin-Funktion nicht unterstützt

2.10 MITTEL: Projekt-Zusammenfassung ignoriert ProjectTimeAllocation

`getProjectSummary` (Zeile 207) filtert `TimeEntries` nach `projectId`:

```
const where: any = { projectId };
```

Dadurch werden nur Einträge erfasst, bei denen das Projekt beim Clock-In direkt gewählt wurde. Allokationen, die nachträglich über `ProjectTimeAllocation` zugewiesen wurden, werden **nicht berücksichtigt**.

2.11 NIEDRIG: Ineffizienter getProjectTimeByUser-Report

`getProjectTimeByUser` (Zeile 443) iteriert über **alle aktiven User x alle aktiven Projekte** und macht für jede Kombination eine separate Datenbankabfrage. Bei 100 Usern und 50 Projekten entstehen 5'000 Queries.

2.12 NIEDRIG: Neue PrismaClient-Instanzen in jedem Controller

Jeder Controller erstellt seine eigene `const prisma = new PrismaClient()`. Dies führt zu:

- Mehreren Connection-Pools
- Potenziellem Connection-Leak bei hoher Last
- Inkonsistenz (kein einheitliches Logging/Middleware)

3. Zusammenfassung der Inkonsistenzen

3.1 userId vs. employeeId Matrix

Komponente	Verwendet	Korrekt?
<code>time.controller.ts</code> – Clock-In/Out	<code>employeeId</code>	✓
<code>time.controller.ts</code> – Pause	<code>employeeId</code>	✓
<code>time.controller.ts</code> – <code>getMyTimeEntries</code>	<code>employeeId</code>	✓
<code>time.controller.ts</code> – <code>getUserTimeEntries</code>	<code>employeeId</code> (via lookup)	✓
<code>time.controller.ts</code> – <code>getLoggedInUsers</code>	<code>employeeId</code>	✓
<code>projectTimeAllocation.controller.ts</code>	<code>employeeId</code> (via lookup)	✓
<code>compliance.service.ts</code>	<code>employeeId</code>	✓
<code>budgetUpdate.service.ts</code>	<code>employeeId</code> (für Abfrage)	✓
<code>report.controller.ts</code> – <code>getMySummary</code>	<code>userId</code>	✗
<code>report.controller.ts</code> – <code>getUserSummary</code>	<code>userId</code>	✗
<code>report.controller.ts</code> – <code>getAllUsersSummary</code>	<code>userId</code>	✗

Komponente	Verwendet	Korrekt?
report.controller.ts – getOvertimeReport	userId	✗
report.controller.ts – getProjectTimeByUser	userId	✗
report.controller.ts – getUserTimeBookingsReport	userId	✗
report.controller.ts – getDetailedTimeBookings	userId	✗

3.2 Stundenberechnung Matrix

Komponente	Brutto/Netto	Korrekt?
report.controller.ts – calculateWorkHours	Brutto	⚠ Keine Pausen
report.controller.ts – getDetailedTimeBookings	Netto	✓
report.controller.ts – getUserTimeBookingsReport	Netto	✓
compliance.service.ts – checkWeeklyHoursViolation	Brutto	✗
compliance.service.ts – checkDailyHoursViolation	Brutto	✗
compliance.service.ts – updateOvertimeBalance	Brutto	✗
compliance.service.ts – checkMissingPauseViolation	Netto	✓
budgetUpdate.service.ts	Netto (Pause abgezogen)	✓

3.3 Projekt-Zuordnung Matrix

Komponente	Quelle	Korrekt?
report.controller.ts – getMySummary	TimeEntry.projectId	⚠ Ignoriert Allokationen
report.controller.ts – getUserSummary	TimeEntry.projectId	⚠ Ignoriert Allokationen
report.controller.ts – getProjectSummary	TimeEntry.projectId	⚠ Ignoriert Allokationen
report.controller.ts – getProjectTimeByUser	TimeEntry.projectId	⚠ Ignoriert Allokationen
report.controller.ts – getDetailedTimeBookings	Beide (mit Allokationen)	✓
report.controller.ts – getUserTimeBookingsReport	Beide (mit Allokationen)	✓
budgetUpdate.service.ts	TimeEntry.projectId	✗ Ignoriert Allokationen
projectTimeAllocation.controller.ts	ProjectTimeAllocation	✓

4. Empfehlungen

4.1 Sofort-Massnahmen (Kritisch)

1. **Report Controller auf `employeeId` migrieren:** Alle Abfragen im Report Controller von `userId` auf `employeeId` umstellen (analog zum Time Controller). Das `userId`-Feld in TimeEntry sollte mittelfristig entfernt werden.
2. **`calculateWorkHours` um Pausenabzug erweitern:** Die Helper-Funktion muss `pauseMinutes` berücksichtigen:

```
const calculateWorkHours = (clockIn: Date, clockOut: Date | null,
  pauseMinutes?: number): number => {
  if (!clockOut) return 0;
  const brutto = (clockOut.getTime() - clockIn.getTime()) / (1000 * 60 * 60);
  return brutto - ((pauseMinutes || 0) / 60);
};
```

3. **Compliance-Service Pausenabzug:** `checkWeeklyHoursViolation`, `checkDailyHoursViolation` und `updateOvertimeBalance` müssen `pauseMinutes` in die Berechnung einbeziehen.

4.2 Kurz-/Mittelfristig (Hoch)

4. **Projekt-Zuordnung vereinheitlichen:** Entscheidung treffen, ob `TimeEntry.projectId` das "Standard-Projekt" ist und `ProjectTimeAllocation` die "echte" Verteilung. Reports und Budget-Updates sollten primär `ProjectTimeAllocation` verwenden, wenn vorhanden.
5. **Budget-Update für Allokationen anpassen:** `budgetUpdate.service.ts` sollte `ProjectTimeAllocation`-Einträge berücksichtigen und die Stunden entsprechend auf die jeweiligen Projekt-Budgets verteilen.
6. **Audit-Trail für Löschungen:** Admin-Löschungen sollten:
 - Over Soft-Delete erfolgen (oder zumindest geloggt werden via `actionService`)
 - Budget und OvertimeBalance rückrechnen
7. **Überlappungs-Validierung:** Bei manuellen Einträgen und Updates prüfen, ob sich Zeiträume für denselben Mitarbeiter überschneiden.

4.3 Langfristig (Mittel/Niedrig)

8. **CostCenter-Integration:** Entweder vollständig implementieren oder das Feld entfernen.
9. **Prisma Client Singleton:** Einen zentralen Prisma Client verwenden (z.B. über `lib/prisma.ts`) statt in jedem Controller eine neue Instanz zu erstellen.
10. **Report-Performance:** `getProjectTimeByUser` über Aggregations-Queries oder materialisierten Views optimieren.
11. **Compliance-Violations updaten statt ignorieren:** Bei erneutem Clock-Out in derselben Woche die bestehende `MAX_WEEKLY_HOURS`-Violation mit dem neuen Wert aktualisieren.

5. Betroffene Dateien

Datei	Pfad	Problem-Kategorien
-------	------	--------------------

Datei	Pfad	Problem-Kategorien
Report Controller	<code>backend/src/controllers/report.controller.ts</code>	userId/employeeId, Pausenberechnung, Allokationen
Time Controller	<code>backend/src/controllers/time.controller.ts</code>	CostCenter, Lösch-Audit
Compliance Service	<code>backend/src/services/compliance.service.ts</code>	Pausenberechnung
Budget Service	<code>backend/src/services/budgetUpdate.service.ts</code>	Allokationen ignoriert
Prisma Schema	<code>backend/prisma/schema.prisma</code>	DEPRECATED userId-Felder
Allocation Controller	<code>backend/src/controllers/projectTimeAllocation.controller.ts</code>	– (korrekt implementiert)